

SIMULACION MULTIAGENTE DE BUSQUEDA Y RESCATE

FLASH POINT FIRE RESCUE

Sergio Escudero Rodriguez
A01253991

David Alejandro Robles Camacho
A01277315

Facundo Gael Pineiro Gonzalez
A01666626

QUE ESTAMOS SIMULANDO

Juego de mesa, en la que seis bomberos entran a un edificio en llamas a sacar victimas antes de que se caiga. Se apaga fuego, se rompen paredes, se cargan victimas y se sacan por las salidas.

6 x 8

celdas del tablero

6

bomberos

24

puntos de daño y colapsa edificio

OBJETIVO

Sacar 7 victimas, para perder se necesitan 4 victimas muertas o si el edificio colapsa.

QUE CONSTRUIMOS

Dos estrategias sobre el mismo modelo en Python con Mesa, un servidor HTTP, y la visualización en Unity.

UNSC-FLASHPOINT

La UNSC-FlashPoint era una nave de reconocimiento arqueológico. En su ultima investigación trajeron algo que llevaba miles de años esperando: **El Flood**, un parásito que se une a cualquier forma de vida

Primero se esconde y junta masa (**el humo**), luego ataca (**el fuego**). Para cuando llega el escuadrón, ya había diez formas activas.

El **Escuadrón Ascu**a, conformado por seis Spartans, entra a sacar a diez científicos antes de que se active el protocolo de contención.

EN LA HISTORIA

EN EL CODIGO

Flood en forma pasiva

humo

Flood activo

fuego

Ecos de los sensores

falsas alarmas (5 de 15 marcadores)

Protocolo de contencion

colapso del edificio a 24 de daño

El escuadron se divide

dos bomberos rescatan, cuatro contienen

INICIO

El bombero aleatorio arma la lista de todo lo que puede hacer con sus puntos de acción y escoge una al azar. Como mover, apagar, abrir una puerta, romper una pared, cargar o dejar una victima.

Regla que no es aleatoria: si va a terminar su turno parado en el fuego, usa su último punto de acción en salirse.

0

de 1000 partidas ganadas

0.04

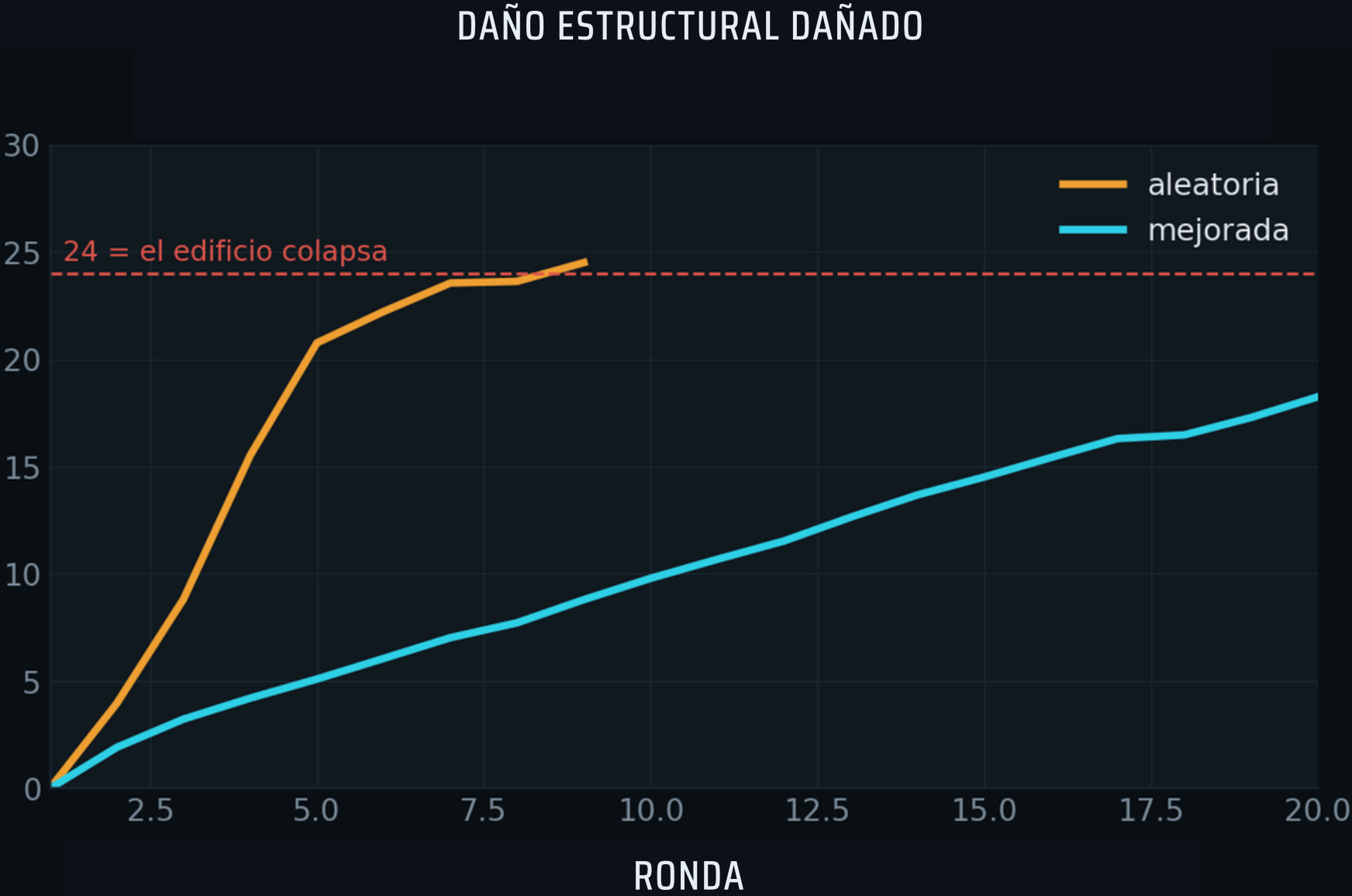
victimas por partida

946

de cada 1000 se le cae el edificio

Sirve como punto de comparación.
Todo lo que venga después se compara con esto.

PUNTOS DE DAÑO



Daño estructural acumulado por ronda, comparando la estrategia aleatoria y la mejorada. La línea roja marca el umbral de colapso del edificio (24 puntos).

EL 29% DEL TABLERO NO EXISTIA

34/48

celdas alcanzables desde
las cuatro salidas

Sin abrir puertas, Dijkstra solo veia 34 de 48 celdas



S = salida X = inalcanzable línea amarilla = puerta cerrada

Las ocho puertas del tablero arrancan cerradas. Mi función de vecinos preguntaba únicamente si se podía pasar, y una puerta cerrada no deja pasar. Para Dijkstra, la celda del otro lado **no existía**.

Un marcador que apareciera en las otras catorce se quemaba sin que nadie pudiera ir. Un fuego que prendiera ahí no se apagaba nunca.

El bombero aleatorio si abría puertas, porque las tenía en su lista de acciones y tarde o temprano le tocaba al azar. La estrategia inteligente estaba jugando con menos tablero que la que no piensa.

DOS REGLAS DEL JUEGO QUE NO ESTABAMOS USANDO

1 ABRIR PUERTAS

El vecindario ahora si cuenta la celda detras de una puerta cerrada, y antes de dar el paso el bombero gasta un punto de accion en abrirla. De 34 celdas alcanzables a las 48.

2 ROLES DINAMICOS

Ahora un rescatista se pasa a contencion mientras haya mas de 8 celdas en llamas, y regresa cuando baja.

El 8 salio de probar seis umbrales con las mismas semillas.

3 EL ORDEN DE LOS TURNOS

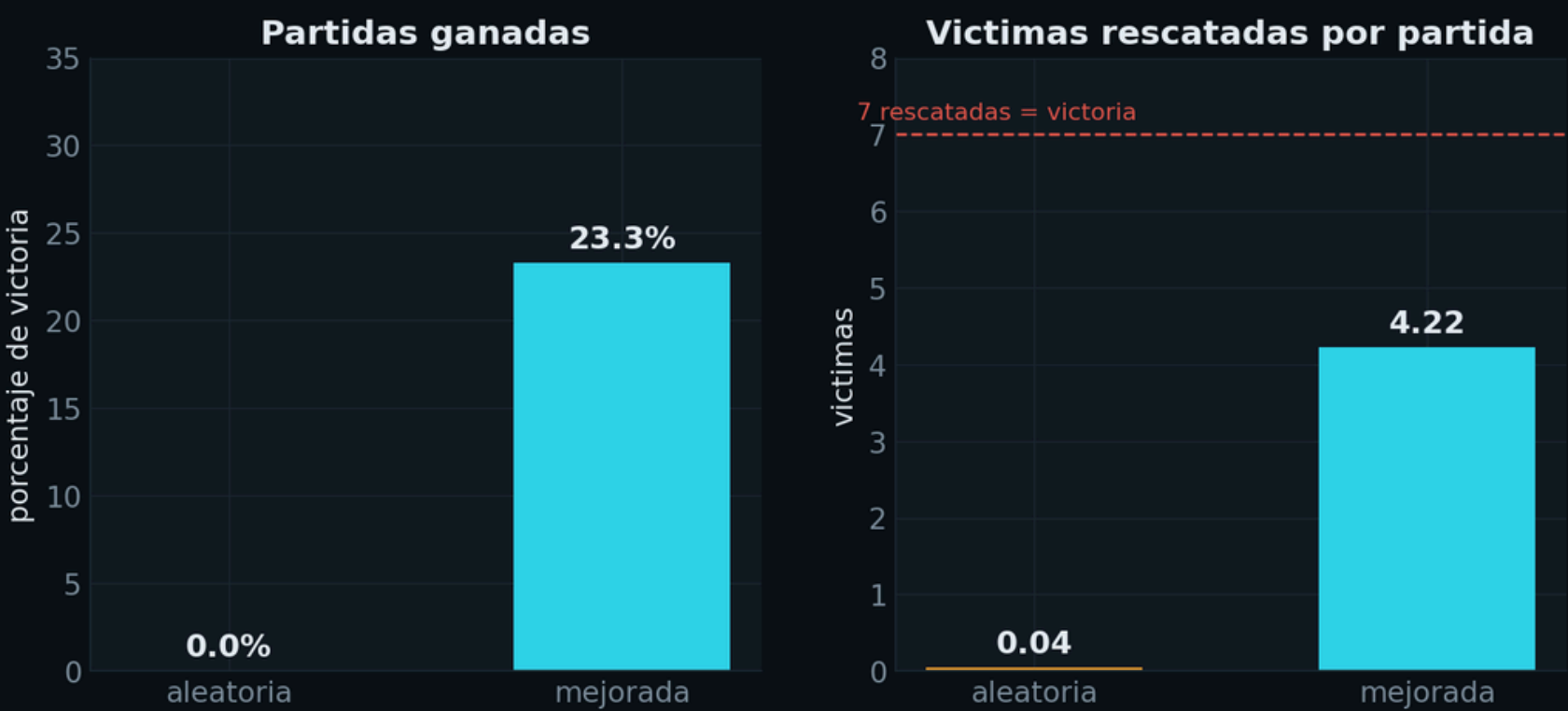
Corriamos los seis turnos de bombero y hasta despues avanzabamos el fuego seis veces seguidas. Lo medimos con los mismos algoritmos y estructura y practicamente nos estaba regalando victorias. Lo pasamos a intercalado y volvimos a correr, probar y ajustar.

Las dos primeras se necesitan entre si. Abrir puertas sin roles dinamicos sube los rescates pero tambien el daño, porque el fuego se propaga por las puertas que abriste.

1000 PARTIDAS POR ESTRATEGIA

	ALEATORIA	MEJORADA
Partidas ganadas	0 de 1000	233 de 1000
Victimas rescatadas	0.04	4.22
Daño al edificio	24.53	18.00
Celdas en llamas	26.71	10.85
Puertas abiertas	3.22	7.46
Rondas que aguanta	4.94	13.13

23.3% de 1000 partidas ganadas por la mejorada



Mismo modelo, mismo tablero, mismas semillas: lo unico que cambia entre las dos columnas es la clase del agente.

Las puertas abiertas son la prueba del cambio: la mejorada abre mas del doble porque las necesita para llegar; la aleatoria las abre por accidente.

CUANTOS BOMBEROS CONVIENE MANDAR A RESCATAR



ARRANCAN DE RESCATE	VICTORIA	MARGEN
2	26.3%	+/- 5.0
3	15.7%	+/- 4.1
4	2.7%	+/- 1.8

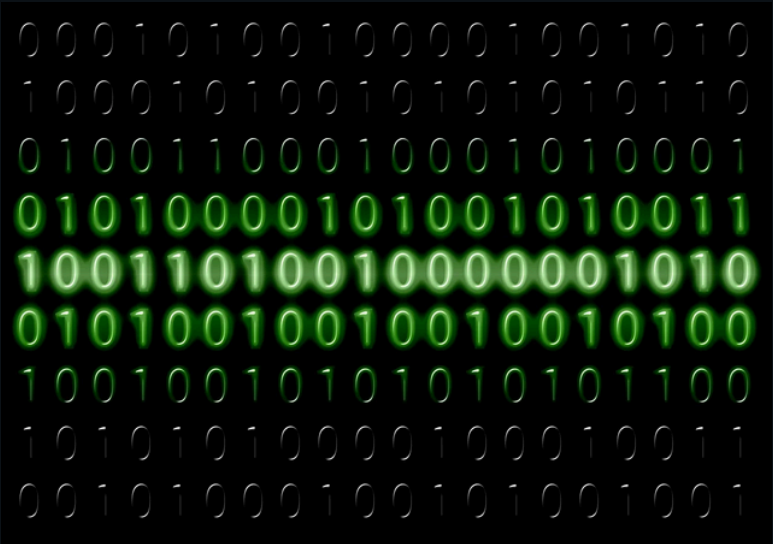
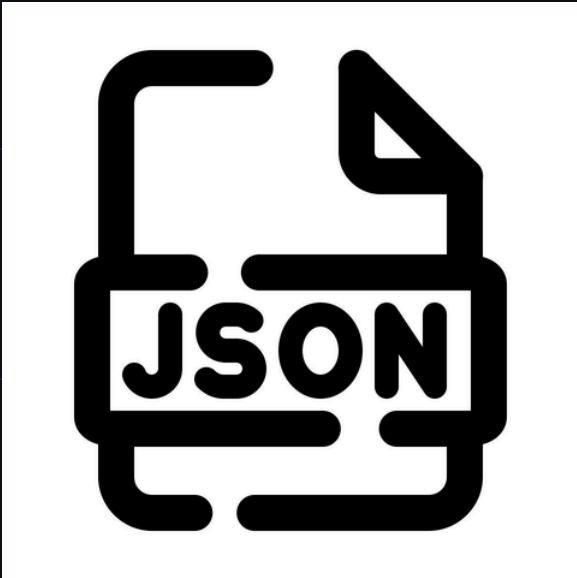
300 partidas por reparto.

Rendimiento: 0.26 segundos por partida, 2900 partidas corridas en total. Por eso descartamos elegir el marcador por costo de rescate completo: mejoraba poquito y hacia un Dijkstra por cada marcador contra cada salida en cada decision.

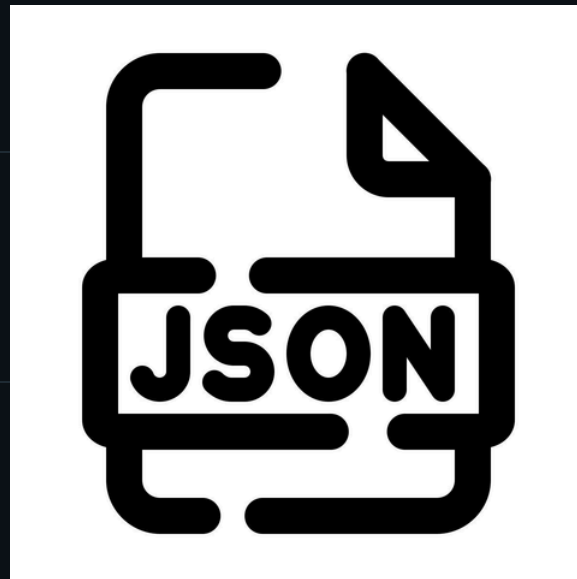
Entre menos bomberos mandas a rescatar, mas victimas rescatas. Con cuatro rescatando solo quedan dos apagando, el fuego se desboca y el edificio colapsa antes de que de tiempo de sacar a nadie.

COMUNICACIÓN DE PYTHON CON UNITY

Diferencias entre nuestra implementación y otras investigadas



COMUNICACIÓN DE PYTHON CON UNITY



Serializador.py

json

EstadoData.cs

JsonUtility



server.py

http.server

WebClient.cs

UnityWebRequest

COMUNICACIÓN DE PYTHON CON UNITY

PROBLEMA

JsonUtility no lee arreglos anidados

No bloquear el juego mientras se espera la respuesta del servidor

Mantener la continuidad entre peticiones

SOLUCIÓN

Aplanar: 48 valores para fuego y poi, 192 para paredes

Corutinas de C# (IEnumerator + yield return SendWebRequest()) la petición corre en paralelo sin congelar el frame

El mismo modelo vive en el servidor y avanza con model.step()

COMO FUNCIONA

Un GET crea partida nueva, un POST corre una ronda y devuelve el estado. Servidor armado sobre el template del profe, HTTP plano en el puerto 8585.

EstadoData

WebClient

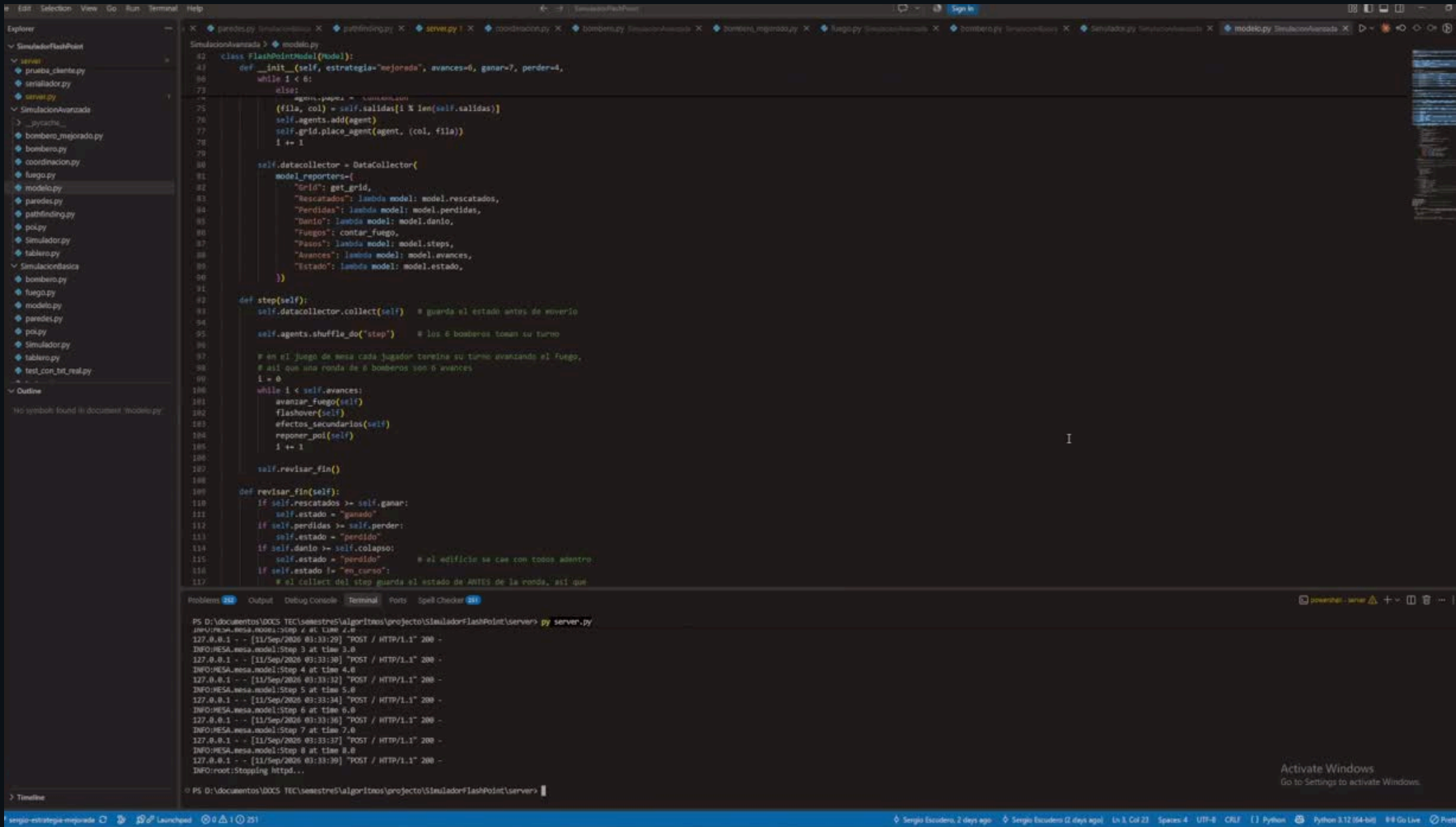
GameManager

Server.py

Serializador.py

VISUALIZACION

EL SISTEMA CORRIENDO

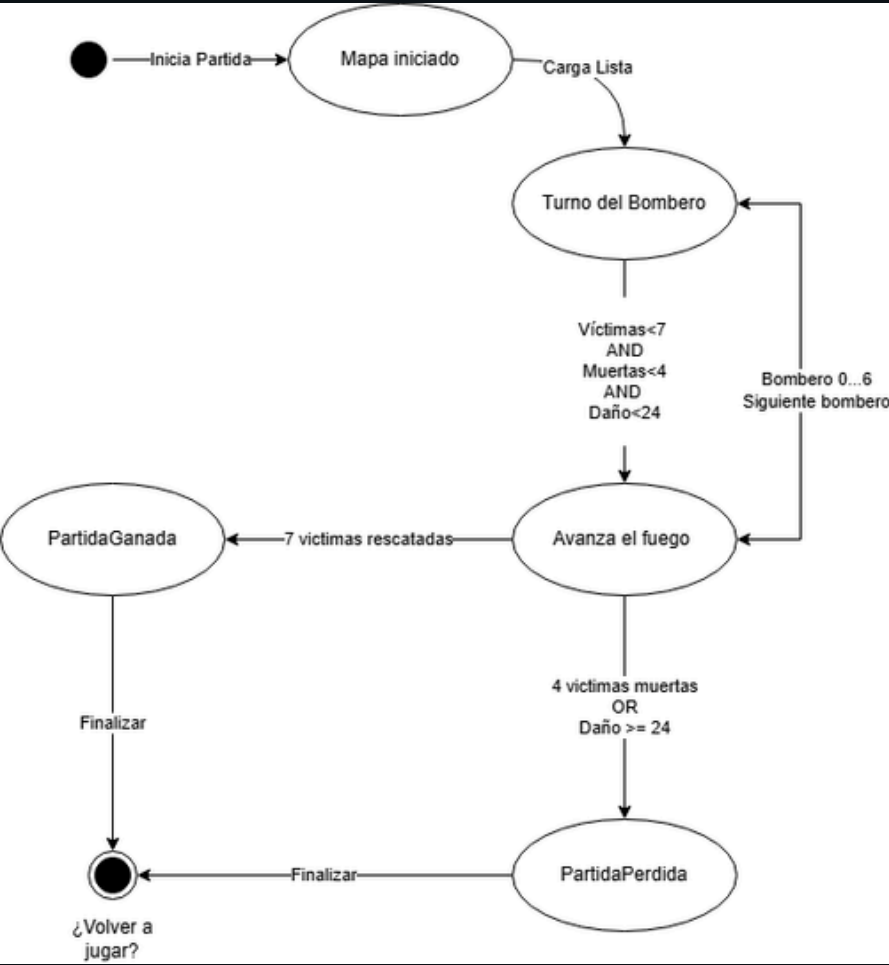


Vista superior 2D con modelos. El HUD es Canvas 2D en Screen Space Overlay.

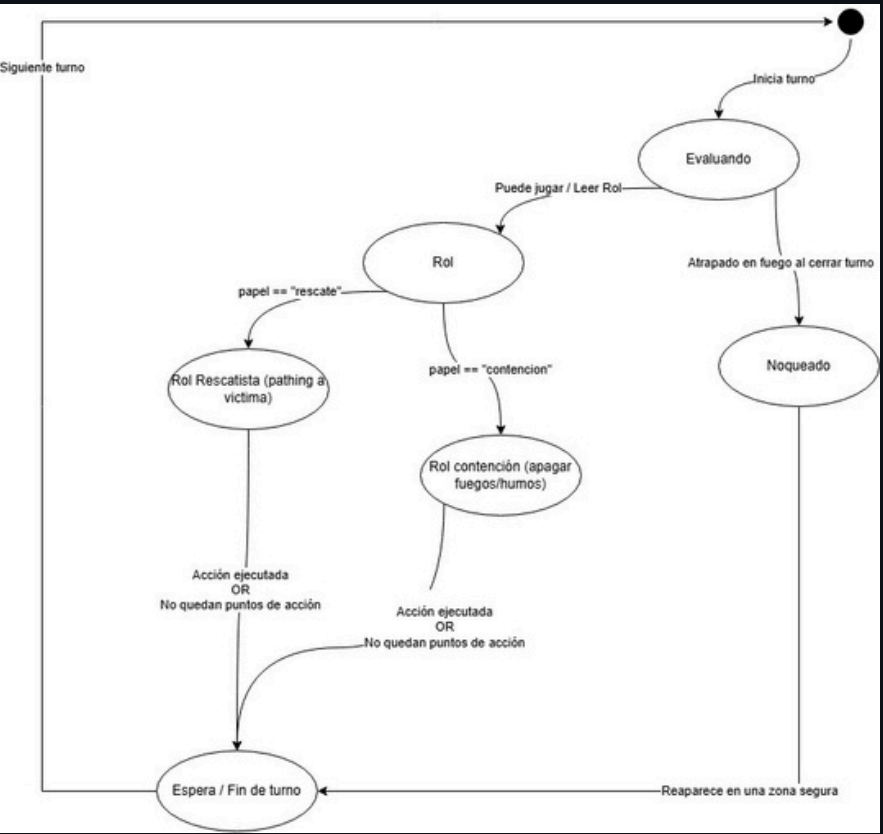
8 puntos de acción por spartan, porque el reglamento deja guardar hasta 4 de un turno a otro.

DIAGRAMAS DE ESTADO

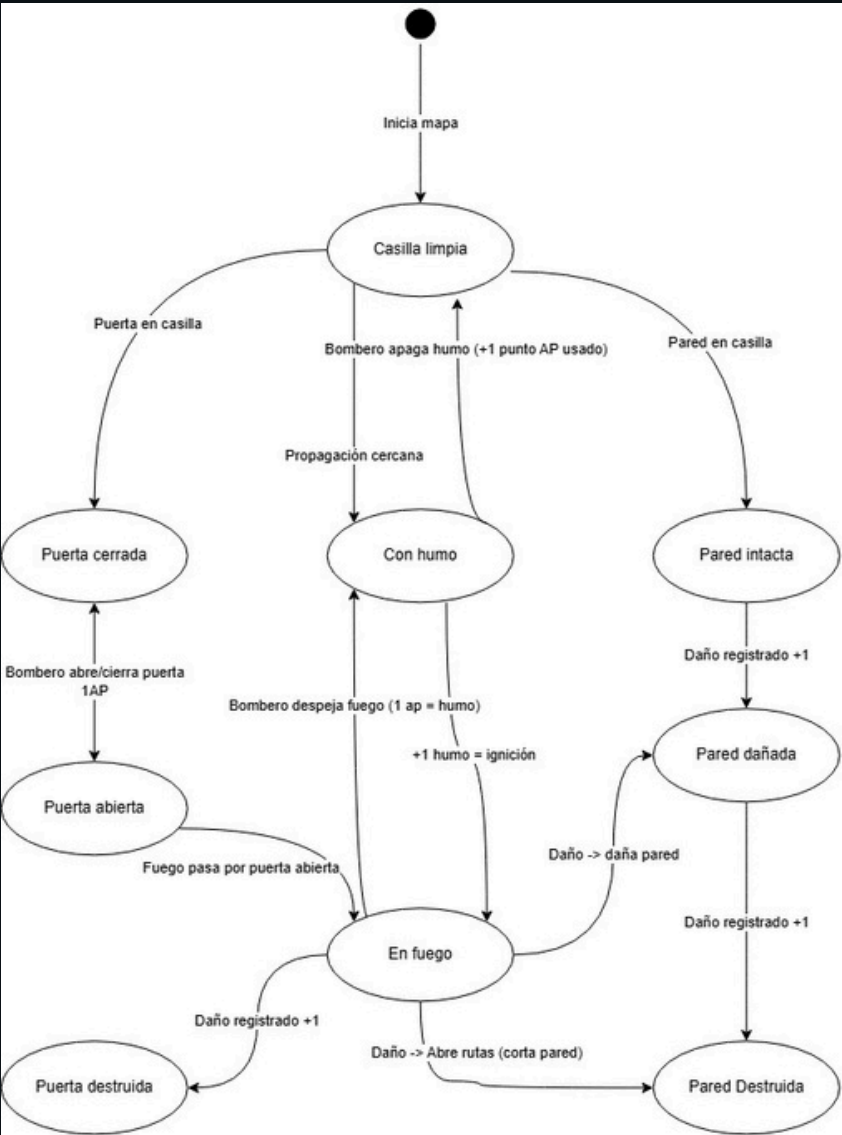
Partida



Bombero



Tablero



QUE SIGUE Y QUE APRENDIMOS

MEJORAS

Que el de contencion apague el fuego que amenaza un marcador, no el que tiene mas cerca. Es el mismo Dijkstra, solo cambia a que celda le pides la ruta.

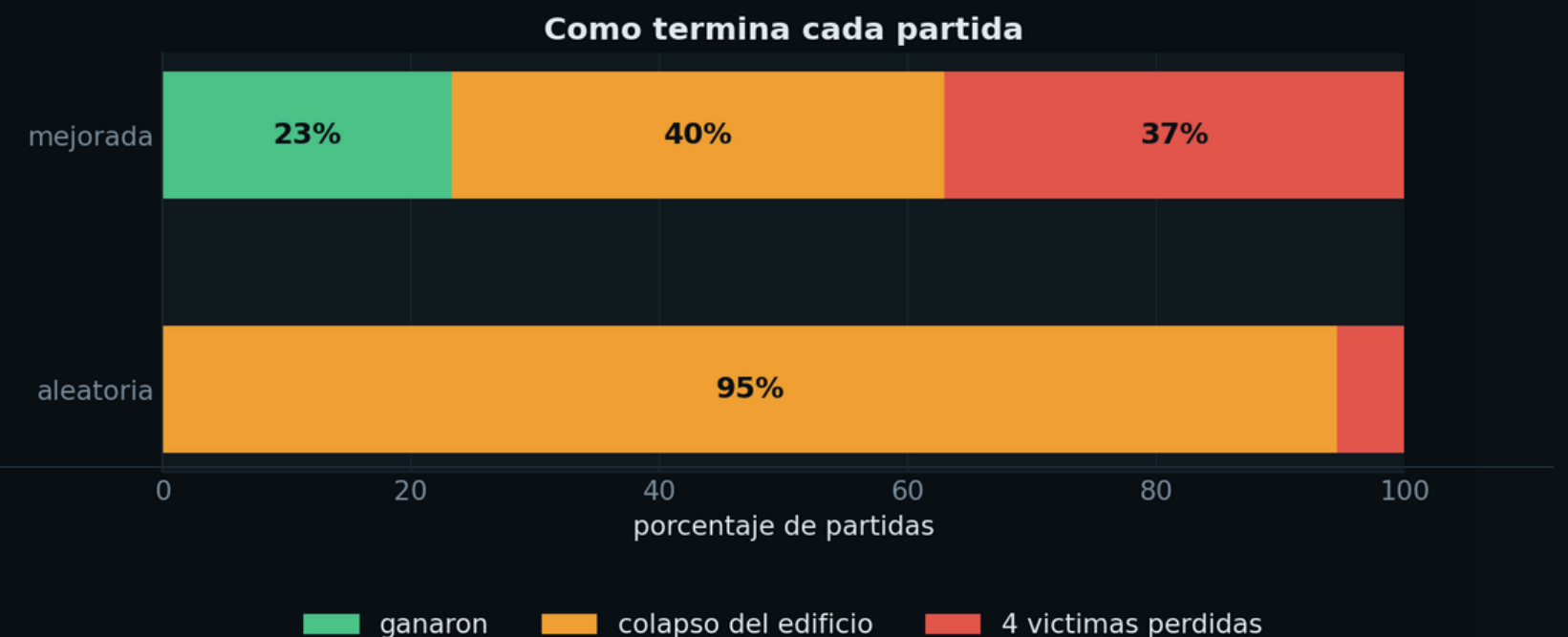
Animaciones de movimiento en Unity. Hoy se borra y se redibuja el tablero completo en cada paso, por eso los bomberos se teletransportan.

233

victorias de 1000, desde 0.
El colapso bajo de 946 a 397 de cada 1000.

CONCLUSIONES

Lo que aplicamos de la materia fue la idea de que el comportamiento del sistema no se programa desde arriba, sale de lo que cada agente decide por su cuenta. Los seis bomberos corren el mismo código y no es que tal cual se hablan entre ellos como si fueran jugadores planeando una estrategia, pero entre todos se reparten el trabajo porque cada uno revisa a dónde va el resto antes de escoger su objetivo. Trabajando todos de manera colaborativa para aumentar las probabilidades de lograr ganar el juego.



Lo que más subió el resultado no fue meter un algoritmo más complicado, fue asegurarnos que estábamos incluyendo todas las reglas y capacidades que necesitaran los agentes